

# PORTAL SATÉLITE DE MONITOREO DE ESTADO DE INSTITUCIONES

Manual del Usuario Final y Especificación Técnica de Arquitectura

|                               |                                                              |
|-------------------------------|--------------------------------------------------------------|
| <b>Sistema Fuente:</b>        | Portal Web BICSA (ASP.NET WebForms / Azure AD SSO)           |
| <b>Solución Desarrollada:</b> | Espejo Satélite de Solo Lectura & Auditoría 24/7             |
| <b>Versión:</b>               | 1.0.0 Producción                                             |
| <b>Fecha de Emisión:</b>      | Agosto 2026                                                  |
| <b>Tecnologías:</b>           | Playwright, Python FastAPI, PostgreSQL, React, Nginx, Docker |

# Tabla de Contenidos

---

## 1. SECCIÓN I: MANUAL DEL USUARIO

- a. Propósito y Alcance de la Solución Satélite
- b. Acceso al Dashboard y Autenticación
- c. Comprensión de Tarjetas KPI y Estados de Instituciones
- d. Motor de Alertas por Horas Hábiles (Límite 72h XML)
- e. Uso de Filtros por Categoría y Buscador en Tiempo Real
- f. Exportación de Reportes a Excel (.xlsx)
- g. Ejecución de Scraping Manual y Consulta de Historial Auditable

## 2. SECCIÓN II: MANUAL TÉCNICO Y ARQUITECTURA

- a. Arquitectura General Monorepo en Docker
- b. Módulo Scraper con Autenticación Azure AD / Microsoft SSO
- c. Diseño de Base de Datos PostgreSQL (Modelos SQLAlchemy)
- d. Lógica de Negocio de Horas Hábiles y Niveles de Alerta
- e. Backend RESTful API (FastAPI) y Generador de Excel
- f. Servicio Frontend SPA (React + Nginx Glassmorphic UI)
- g. Planificación de Corridas Cron (07:00 hs y 16:00 hs)
- h. Guía de Despliegue Local (Windows 11 + Docker Desktop)
- i. Guía de Despliegue en Producción (Oracle Cloud VM ARM64)

# SECCIÓN I: MANUAL DEL USUARIO

## 1.1 Propósito y Alcance de la Solución Satélite

El **Portal Satélite de Monitoreo BICSA** es una plataforma web independiente desarrollada exclusivamente para el **\*\*seguimiento, auditoría y alerta temprana\*\*** sobre el estado operacional de las instituciones financieras registradas en el portal BICSA.

**⚠ Principio de Solo Lectura:** Este sistema satélite actúa como un espejo/monitor externo de consulta. **\*\*NO modifica\*\*** ni altera los datos o estados reales dentro del portal original de BICSA, ya que BICSA administra sus transiciones con su propio proceso nocturno (24/7 a las 00:00 hs).

## 1.2 Acceso al Dashboard y Autenticación

Para ingresar al Dashboard Satélite:

- Abrir el navegador web e ingresar la dirección URL: `http://localhost:3000` (o la dirección IP del servidor en nube).
- Ingresar con las credenciales asignadas para el Dashboard Satélite:
  - **Usuario:** `admin@bicasatelite.com`
  - **Contraseña:** `AdminPassword2026!`

## 1.3 Tarjetas KPI y Estados de Instituciones

En la parte superior del Dashboard se presentan tarjetas de resumen con los indicadores principales:

| Indicador KPI            | Descripción                                                                           |
|--------------------------|---------------------------------------------------------------------------------------|
| <b>Total Registradas</b> | Total de instituciones financieras vigentes obtenidas de la última sincronización.    |
| <b>Activas</b>           | Instituciones operativas que cuentan con carga de XML al día y consultas permitidas.  |
| <b>Bloqueadas</b>        | Instituciones bloqueadas por BICSA por haber superado las 72 horas hábiles sin carga. |
| <b>Suspendidas</b>       | Instituciones suspendidas por inconsistencias administrativas o de formato XML.       |
| <b>Validación XML</b>    | Instituciones en etapa previa/paralela de validación de archivos XML.                 |

## 1.4 Motor de Alertas por Horas Hábiles (Límite 72h)

BICSA exige que cada institución activa realice la carga de su archivo XML dentro de un plazo máximo de **\*\*72 horas hábiles\*\*** (lunes a viernes). Si transcurre dicho plazo sin carga, BICSA cambia el estado a **\*\*Bloqueada\*\***.

El Portal Satélite evalúa en tiempo real el tiempo hábil transcurrido y clasifica la alerta en 3 niveles de color:

● **NORMAL (Verde):** Transcurrieron menos de 48 horas hábiles desde la última carga. La institución se encuentra al día.

● **ADVERTENCIA (Amarillo):** Transcurrieron entre 48 y 72 horas hábiles sin carga. La institución dispone de menos de 24 horas hábiles antes de ser bloqueada.

● **CRÍTICO (Rojo):** Transcurrieron más de 72 horas hábiles sin carga XML. Riesgo inminente de bloqueo por el proceso nocturno de BICSA.

**Exclusión de Instituciones Desvinculadas, Bloqueadas y Suspendidas:** Las instituciones que ya se encuentran en estado *Desvinculada* (contrato cesado), *Bloqueada* o *Suspendida* son excluidas de las alertas de carga preventiva, mostrando badge neutro o su estado correspondiente.

## 1.5 Filtros y Buscador en Tiempo Real

- **Pestañas de Categorías:** Permiten filtrar la tabla por las 7 categorías oficiales: *Todas*, *Activa*, *Suspendida*, *Bloqueada*, *Con excepción de carga*, *Desvinculada*, *Validación de XML*, *Activa (límite de consultas)*.
- **Barra de Búsqueda:** Filtra instantáneamente por el nombre de la institución o por el motivo de suspensión.

## 1.6 Exportación de Reportes a Excel (.xlsx)

Al hacer clic en el botón **"Exportar Excel"** en la barra superior, el sistema genera y descarga un archivo `.xlsx` estilizado profesionalmente, incluyendo encabezados navy, resumen de KPIs y formato de colores condicionales según el nivel de alerta.

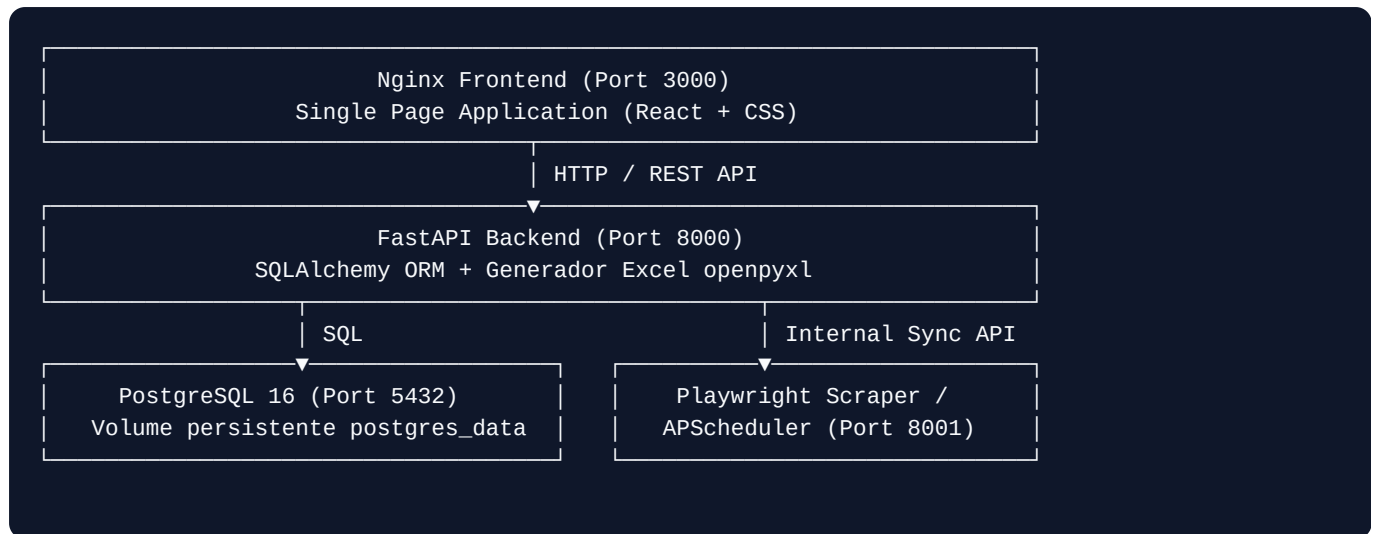
## 1.7 Scraping Manual e Historial Auditable

- **Ejecutar Scraping:** Dispara de inmediato la extracción en vivo desde BICSA sin esperar al cron automático.
- **Historial:** Abre una ventana modal con la bitácora auditable de todos los cambios de estado detectados (con fecha, estado anterior, estado nuevo y origen de la corrida).

## SECCIÓN II: MANUAL TÉCNICO Y ARQUITECTURA

### 2.1 Arquitectura General Monorepo en Docker

El sistema está diseñado como un monorepo compuesto por 4 microservicios desacoplados y orquestados mediante `docker-compose.yml` :



### 2.2 Módulo Scraper con Autenticación Azure AD SSO

El microservicio `scraper` utiliza **Python + Playwright (Headless Chromium)** debido a que la aplicación fuente de BICSA se encuentra protegida por **Microsoft Azure Active Directory (Azure AD / Entra ID SSO)**.

#### Flujo de Autenticación Playwright:

1. Navega a `https://bicquerywebapp.azurewebsites.net` y detecta la redirección hacia `login.microsoftonline.com`.
2. Completa el campo de correo `input[name='loginfmt']` con `BICSA_USER` y presiona el botón Siguiente.
3. Completa el campo de contraseña `input[name='passwd']` con `BICSA_PASSWORD` y presiona Iniciar sesión.
4. Resuelve o salta el diálogo de permanencia de sesión (`idSIButton9`).
5. Navega a la pantalla `Instituciones_Estado.aspx` y extrae el HTML completo.
6. Parse el HTML usando **BeautifulSoup**, limpiando las filas y mapeando dinámicamente las 10 tablas HTML.
7. Envía los datos mediante un **POST** interno al backend a `/api/v1/internal/sync-scrape`.

### 2.3 Modelo de Base de Datos PostgreSQL

Modelos relacionales principales definidos en SQLAlchemy:

- `instituciones` : Catálogo maestro (id, nombre, codigo\_bicsa, creado\_el).
- `estado_actual` : Foto más reciente por institución (con restricción `UNIQUE(institucion_id)`, horas hábiles transcurridas, horas restantes y nivel\_alerta).
- `snapshot_diario` : Registro histórico capturado durante la corrida de las 07:00 hs.
- `historial_cambios` : Eventos auditables de transición de estado (estado\_anterior, estado\_nuevo, fecha\_deteccion, corrida\_origen).
- `usuarios` : Cuentas de acceso local al Dashboard (email, hashed\_password con bcrypt).

## 2.4 Lógica de Negocio de Horas Hábiles

La función `calcular_horas_habiles(fecha_inicio, fecha_fin)` itera día por día evaluando `weekday() < 5` (lunes=0 a viernes=4), descartando sábados y domingos.

```
horas_transcurridas = calcular_horas_habiles(fecha_carga, ahora)
horas_restantes = max(0.0, round(72.0 - horas_transcurridas, 2))

if horas_transcurridas > 72.0:
    nivel_alerta = "CRITICO"
elif horas_transcurridas >= 48.0:
    nivel_alerta = "ADVERTENCIA"
else:
    nivel_alerta = "NORMAL"
```

## 2.5 Programación Cron de Scrapes

- **07:00 hs (Diario):** Corrida `FULL` . Genera el `SnapshotDiario` y registra variaciones en `HistorialCambios` .
- **16:00 hs (Lunes a Viernes):** Corrida `LIGHT` . Refresca únicamente la vista de `EstadoActual` en vivo.

## 2.6 Guía de Despliegue en Producción (Oracle Cloud VM ARM64)

Para compilar las imágenes en arquitectura multi-plataforma (amd64 para Windows y arm64 para Oracle Cloud Ubuntu 22.04):

```
# Crear builder multi-arquitectura
docker buildx create --name multiarch-builder --use
docker buildx inspect --bootstrap

# Compilar e impulsar imágenes
docker buildx build --platform linux/amd64,linux/arm64 -t usuario/bicsa-scraper:latest ./scraper -
docker buildx build --platform linux/amd64,linux/arm64 -t usuario/bicsa-backend:latest ./backend -
docker buildx build --platform linux/amd64,linux/arm64 -t usuario/bicsa-frontend:latest ./frontend
```

En la máquina virtual Oracle Cloud ARM64:

```
git clone https://github.com/tu-repo/docker-portal-estado-institucionesBIC.git
cd docker-portal-estado-institucionesBIC
cp .env.example .env
nano .env # Configurar credenciales reales de BICSA
docker compose up -d
```

## 2.7 Monitoreo y Diagnóstico de Logs

```
# Ver logs del backend
docker logs -f bicsa_backend

# Ver logs del scraper y Playwright
docker logs -f bicsa_scraper

# Ver estado de los contenedores
docker compose ps
```